

Cloud and Network Security.

Friday, July 7th, 2025.

Week 9 Assignment:

Key Vault and Encryption (Azure).

Contents

Introduction	3
Deploy the base infrastructure from an ARM template	3
Deploy an Azure VM and an Azure SQL Database	3
Configure the Key Vault resource with a key and a secret.....	5
Create and configure a Key Vault	5
Configuring Key Vault Access Policies	6
Creating a Key Vault Access Policy	8
Adding a Key to Azure Key Vault	11
Configure an Azure SQL database and a data-driven application.....	12
Enable a client application to access the Azure SQL Database service.	12
Create a policy allowing the application access to the Key Vault.	13
Retrieve SQL Azure database ADO.NET Connection String	14
Log on to the Azure VM running Visual Studio 2019 and SQL Management Studio 19.....	15
Create a table in the SQL Database and select data columns for encryption.....	16
Resource Cleanup	22
Exercise Summary	23
Conclusion.....	24

Introduction

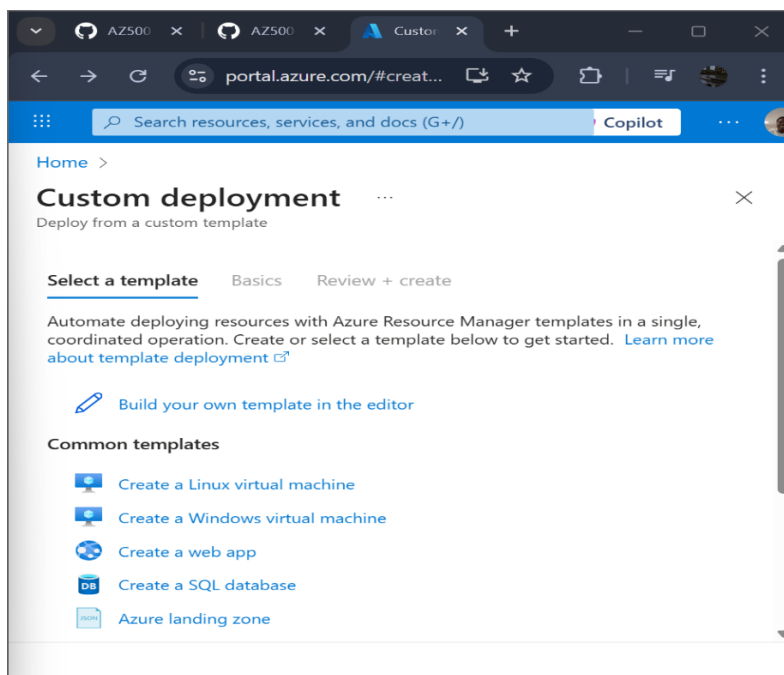
This report documents the steps and outcomes of a practical exercise involving the deployment and management of Azure resources with a specific focus on Azure Key Vault and encryption-related configurations. The purpose of this lab was to simulate real-world cloud administration and security tasks as part of an Azure training assignment, while minimizing incurred costs by managing and cleaning up resources efficiently.

Deploy the base infrastructure from an ARM template

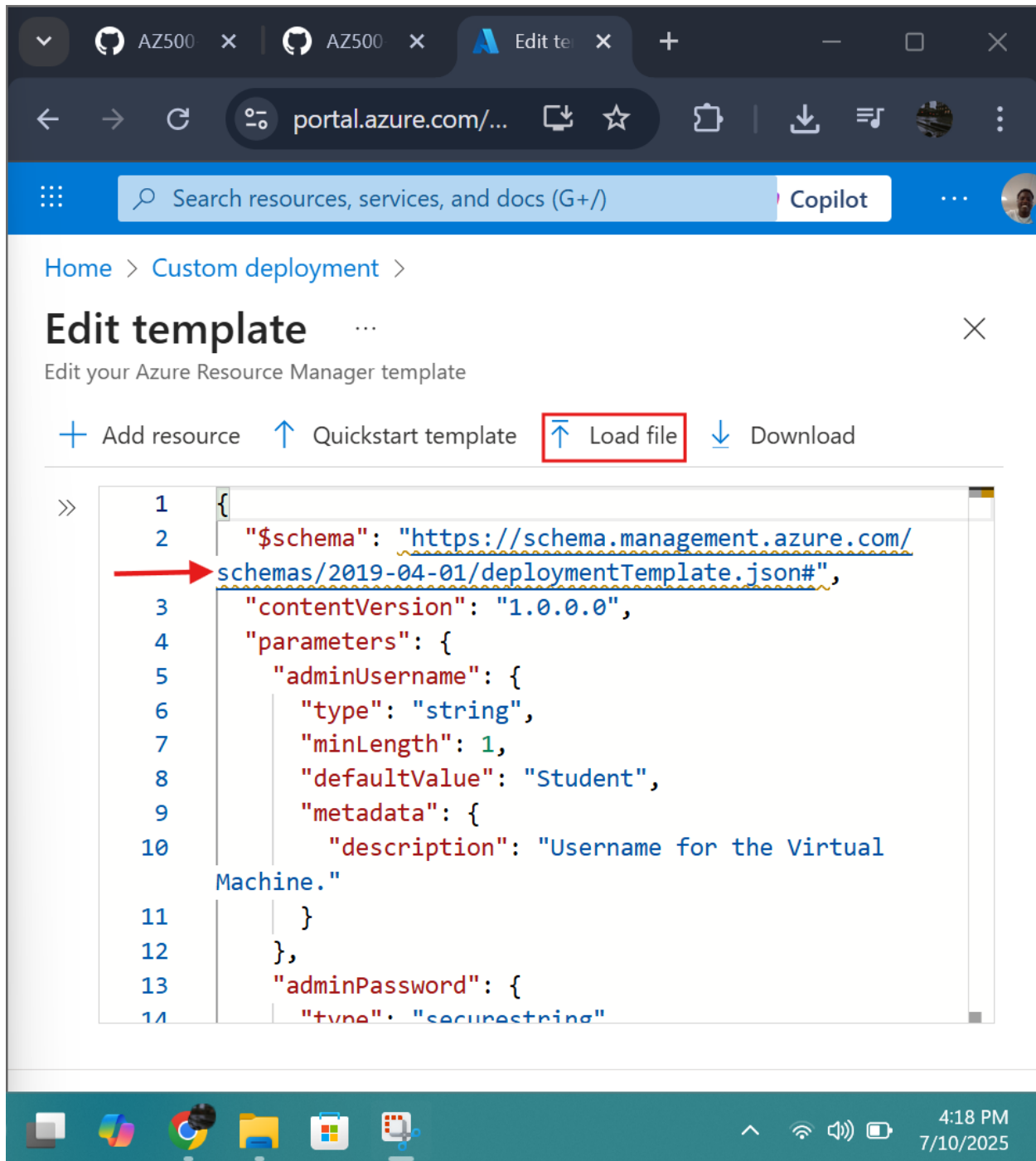
Deploy an Azure VM and an Azure SQL Database

In this task, the objective was to deploy a virtual machine (VM) in Azure pre-configured with Visual Studio 2019 and SQL Server Management Studio 19. Alongside the VM, an Azure SQL Database was also deployed as part of the infrastructure setup for further use in later tasks.

To begin, I signed in to the [Azure portal](#) using an account that holds either **Owner** privileges within the subscription being used. These elevated permissions are required to perform resource deployments via custom templates. After logging in, I used the search bar at the top of the Azure portal interface to search for “**Deploy a custom template**” and pressed Enter. From the resulting options, I selected “**Build your own template in the editor**” under the **Custom deployment** blade.



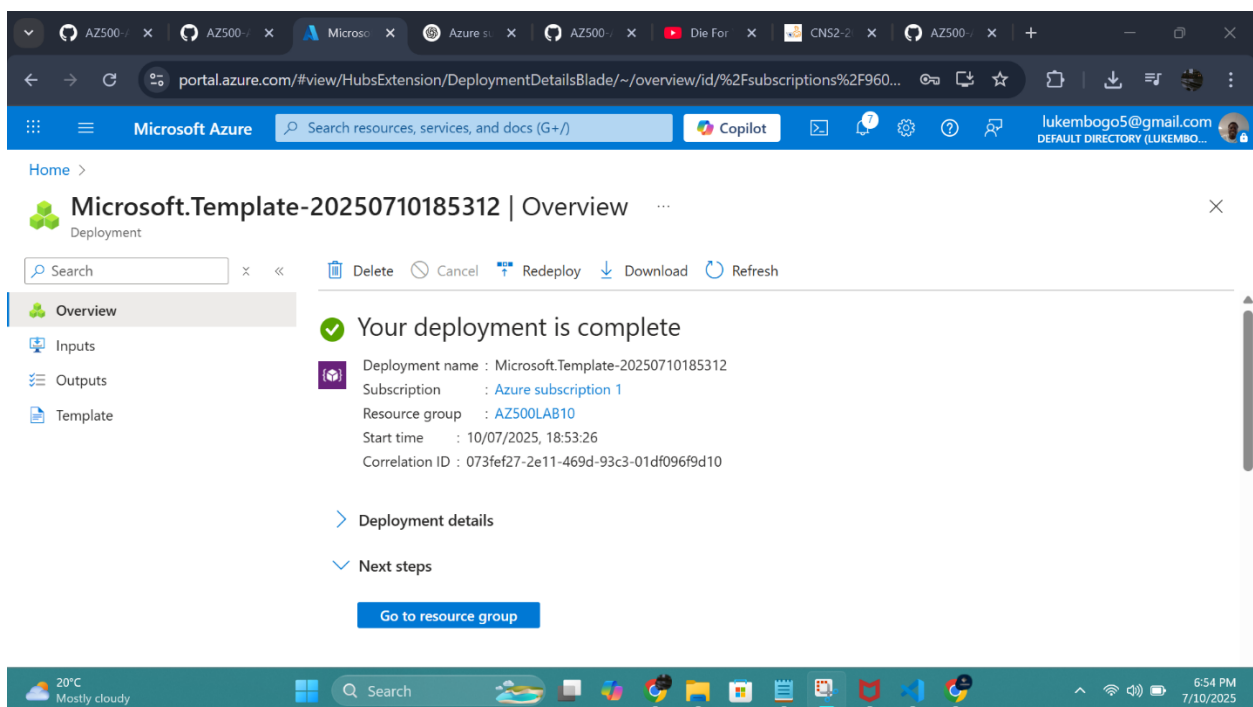
After opening the Edit template blade, I clicked on Load file to upload the deployment template file named azure_vm_deployment_template (10).json from my local machine. Once the file was selected, I clicked Open, and the contents of the JSON template were loaded into the editor.



With the template loaded, I clicked **Save** to proceed to the **Custom deployment** blade, where I configured the required deployment parameters. The following settings were applied:

- **Subscription:** Selected the Azure subscription associated with the lab.
- **Resource group:** Clicked **Create new** and entered the name AZ500LAB10.
- **Location:** Chose **East US** as the deployment region.
- **Username:** Set to Student.
- **Password:** Entered the personal password created earlier in Lab 02, Exercise 2, Task 1.

Although the administrative credentials (username and password) can be changed, I used the default username Student and reused a previously created secure password. I then proceeded to deploy the resources.



Configure the Key Vault resource with a key and a secret

Create and configure a Key Vault

In this task, I created an Azure Key Vault within the previously deployed resource group. The Key Vault will serve as a secure location to store keys, secrets, and certificates used throughout the lab.

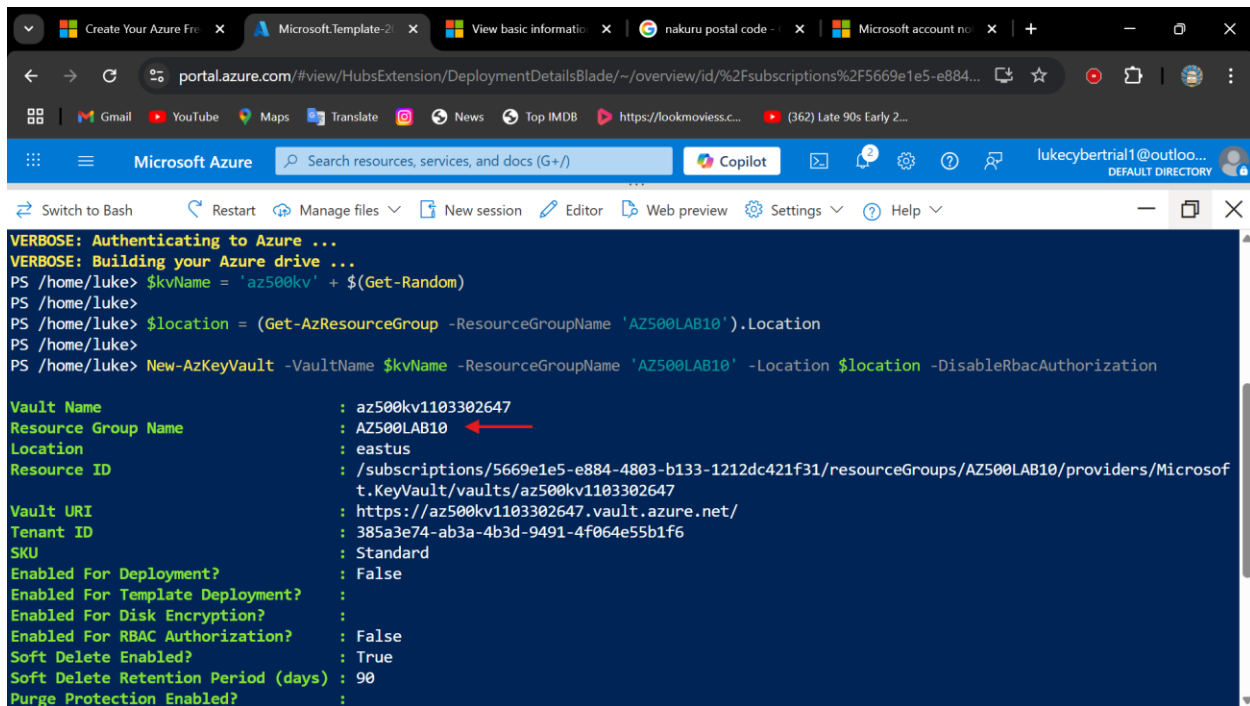
To begin, I accessed **Azure Cloud Shell** using the terminal icon at the top of the Azure Portal. I selected **PowerShell** as the environment and ensured a storage account was created if prompted.

Once the shell was ready, I defined two variables: one for the Key Vault name (\$kvName) and another for the location pulled from the existing resource group AZ500LAB10. The Key Vault name was generated dynamically to ensure global uniqueness. I used the following commands [*\$kvName = 'az500kv' + \$(Get-Random)*

\$location = (Get-AzResourceGroup -ResourceGroupName 'AZ500LAB10').Location

New-AzKeyVault -VaultName \$kvName -ResourceGroupName 'AZ500LAB10' -Location \$location -DisableRbacAuthorization].

This command successfully provisioned the Key Vault while disabling RBAC authorization, meaning access would be controlled using **access policies** instead. The output of the last command will displayed the vault name and the vault URI. The vault URI is in the format



```
VERBOSE: Authenticating to Azure ...
VERBOSE: Building your Azure drive ...
PS /home/luke> $kvName = 'az500kv' + $(Get-Random)
PS /home/luke>
PS /home/luke> $location = (Get-AzResourceGroup -ResourceGroupName 'AZ500LAB10').Location
PS /home/luke>
PS /home/luke> New-AzKeyVault -VaultName $kvName -ResourceGroupName 'AZ500LAB10' -Location $location -DisableRbacAuthorization

Vault Name                : az500kv1103302647
Resource Group Name       : AZ500LAB10
Location                  : eastus
Resource ID               : /subscriptions/5669e1e5-e884-4803-b133-1212dc421f31/resourceGroups/AZ500LAB10/providers/Microsoft.KeyVault/vaults/az500kv1103302647
Vault URI                 : https://az500kv1103302647.vault.azure.net/
Tenant ID                 : 385a3e74-ab3a-4b3d-9491-4f064e55b1f6
SKU                       : Standard
Enabled For Deployment?   : False
Enabled For Template Deployment? : False
Enabled For Disk Encryption? : False
Enabled For RBAC Authorization? : False
Soft Delete Enabled?      : True
Soft Delete Retention Period (days) : 90
Purge Protection Enabled? : 
```

Configuring Key Vault Access Policies

After successfully deploying the Key Vault, I proceeded to configure access permissions to allow selected users or applications to perform key management operations.

From the **Azure portal**, I navigated to the **Resource groups** section by entering “*Resource groups*” in the search bar and selecting it from the results. In the list of resource groups, I selected **AZ500LAB10**, which was created earlier during the VM deployment phase.

The screenshot shows the Microsoft Azure portal's 'Resource groups' page. The page title is 'Resource groups' with a subtitle 'Default Directory'. Below the title, there are action buttons: '+ Create', 'Manage view', 'Refresh', 'Export to CSV', 'Open query', and 'Assign tags'. A filter bar shows 'Subscription equals all' and 'Location equals all'. The main table lists resource groups with columns for 'Name', 'Subscription', and 'Location'. The 'AZ500LAB10' resource group is highlighted with a red arrow pointing to its 'Location' column, which shows 'East US'.

Name	Subscription	Location
AZ500LAB10	Azure subscrip...	East US
NetworkWatcherRG	Azure subscrip...	East US

Showing 1 - 2 of 2. Display count: 10

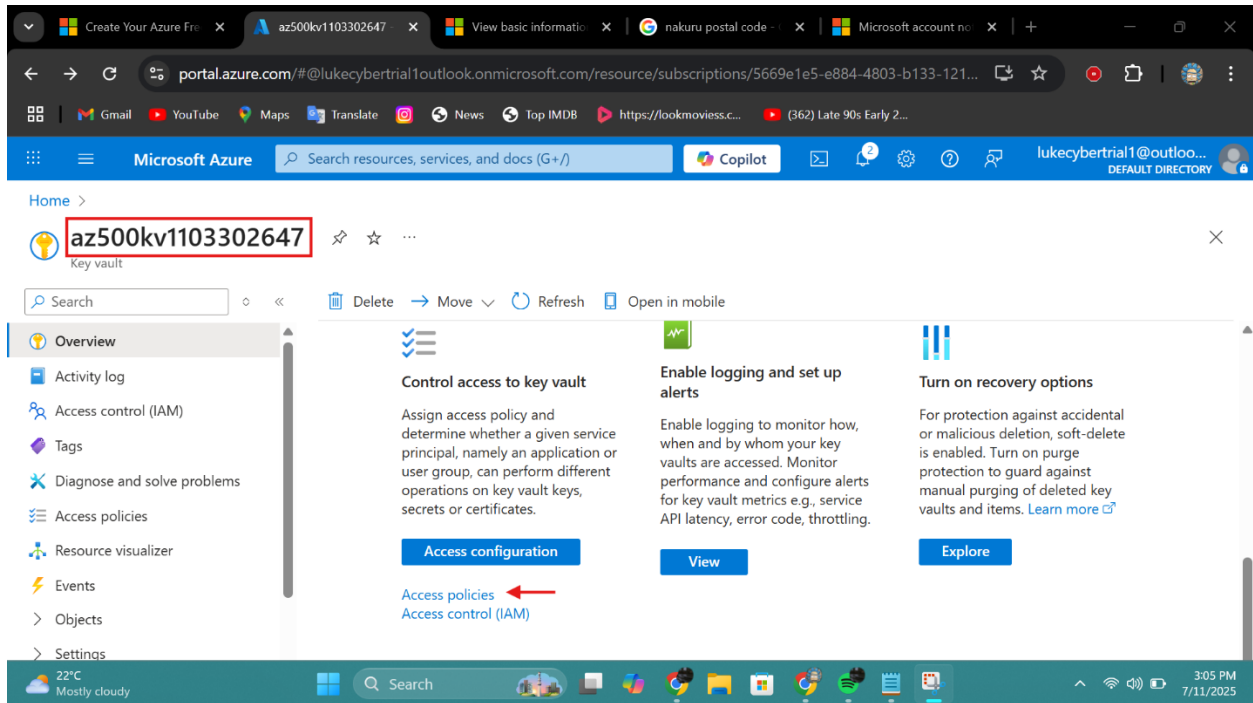
Within this resource group, I located and clicked on the **Key Vault** resource that was recently provisioned. On the **Key Vault blade**, I accessed the **Overview** section and selected **Access policies**.

From the Access policies blade, I clicked the **+ Create** button to initiate the process of assigning specific permissions.

The screenshot shows the Microsoft Azure portal's 'AZ500LAB10' resource group page. The 'Overview' section is selected in the left-hand navigation pane. The main area displays a table of resources within the resource group. A red arrow points to the 'az500kv1103302647' resource, which is a 'Key vault' located in 'East US'.

Name	Type	Location
az500-10-nsg1	Network security group	East US
az500-10-pip1	Public IP address	East US
az500-10-vm1	Virtual machine	East US
az500-10-vm1_OsDisk_1_5f82f31e464940849203739a33...	Disk	East US
az500-10-vnet1	Virtual network	East US
az500kv1103302647	Key vault	East US
vhdstoragehxxdjakrwip3u	Storage account	East US

Page 1 of 1



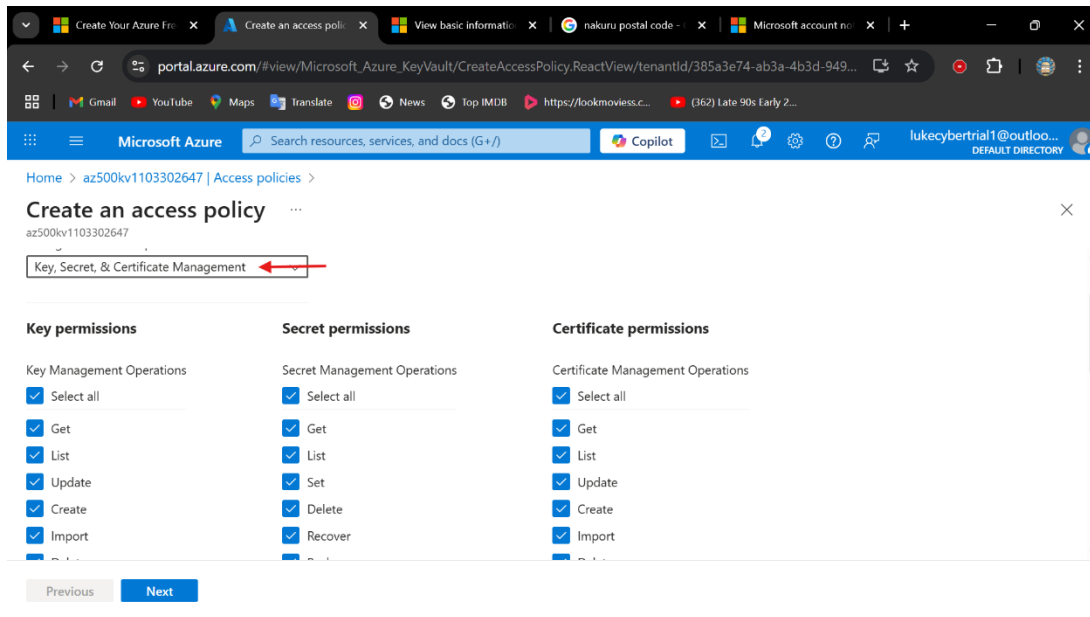
Creating a Key Vault Access Policy

After clicking the **+ Create** button in the Access policies section, the **Create an access policy** blade appeared, where I configured access control for my user account.

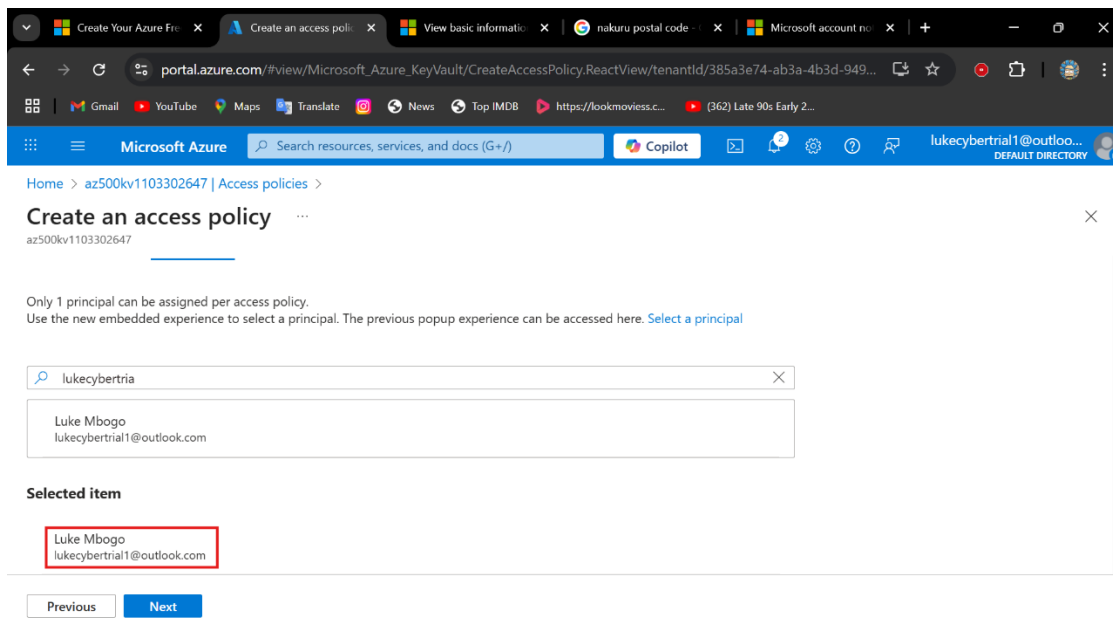
I used the following settings:

- **Configure from template (optional):**
Selected **Key, Secret, & Certificate Management**.
- **Key permissions:**
Clicked **Select all** (resulting in a total of 9 permissions selected).
- **Key permissions / Cryptographic Operations:**
Additionally, selected **Sign** (making it 1 cryptographic permission selected).
- **Secret permissions:**
Clicked **Select all** (7 permissions selected).
- **Certificate permissions:**
Clicked **Select all** (15 permissions selected).
- **Select principal:**
Clicked on **None selected**, then from the **Principal blade**, chose my **user account**, and clicked **Next**.

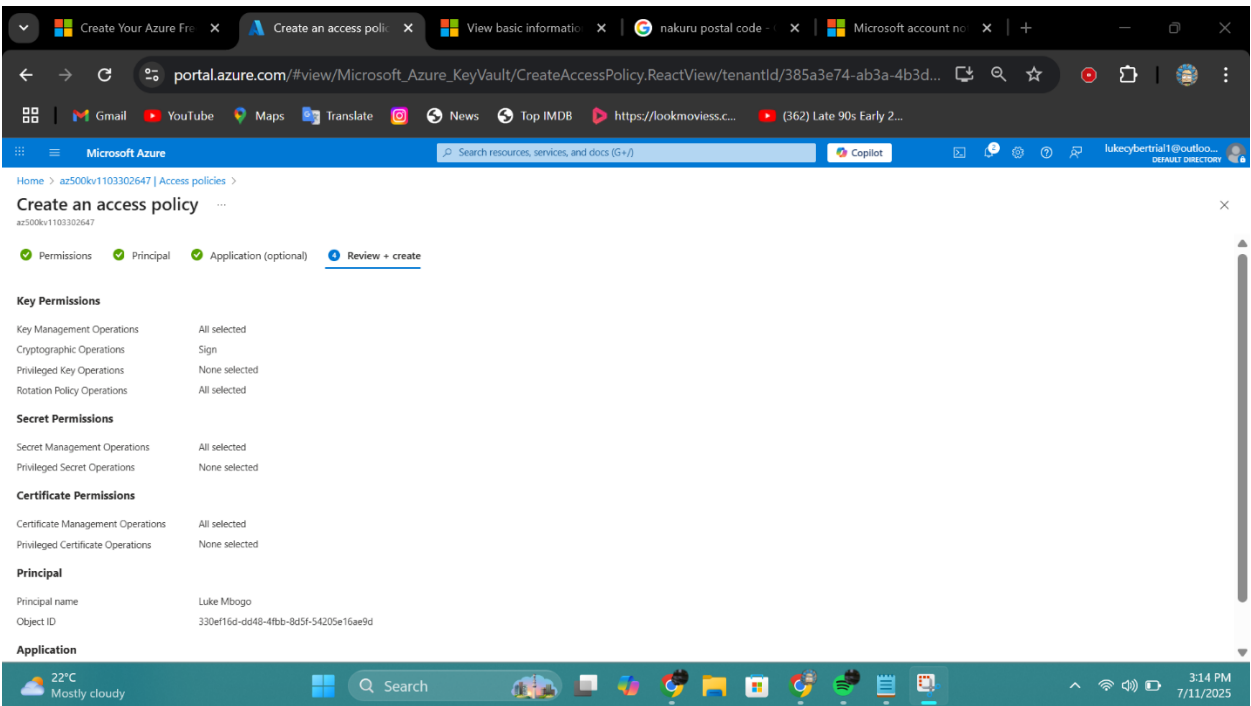
- **Application (optional):**
Skipped by clicking **Next**.
- **Review + create:**
Reviewed the configuration and clicked **Create**.



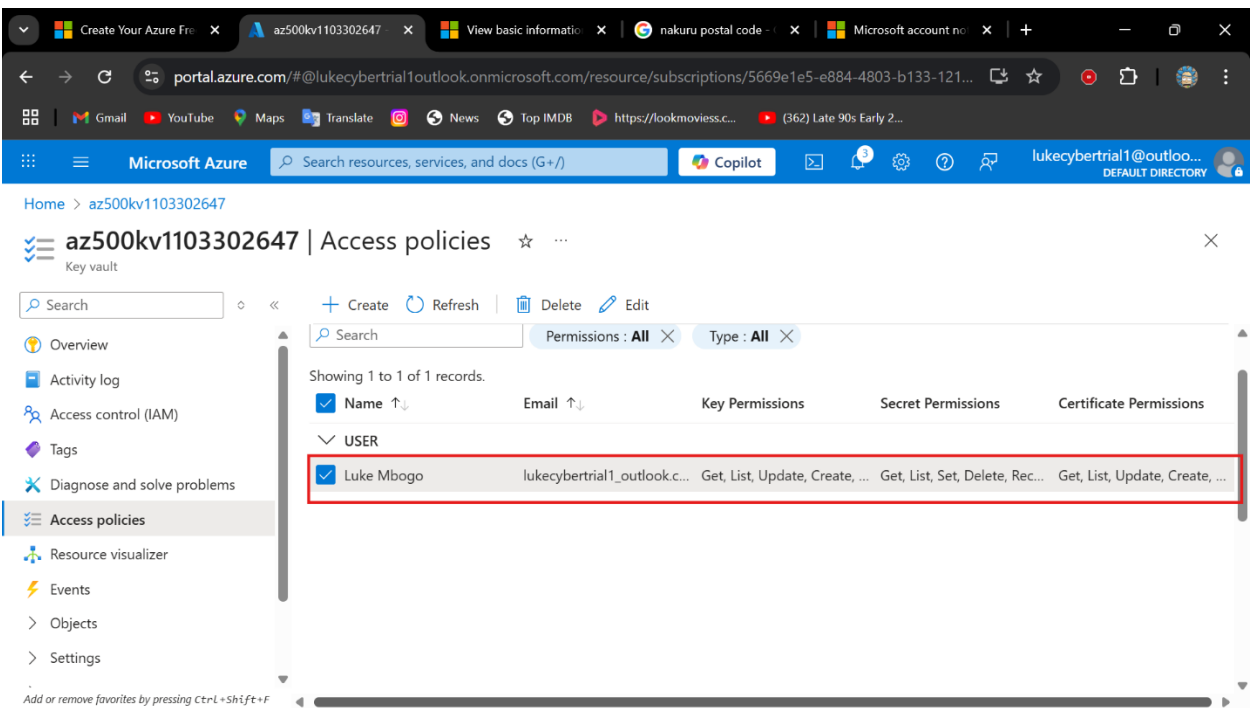
I then selected my account as the principle for the access policy



I proceeded to the review and create page where I confirmed all my configurations that I had made the after proceeded to click on create, this is for validating all the configurations.



After creation, the screen returned to the **Access policies** page where the configured policy appeared, listing the **Application**, **Email**, **Key Permissions**, **Secret Permissions**, and **Certificate Permissions**.



Adding a Key to Azure Key Vault

In this task, I added a software-protected key to the Azure Key Vault and verified its creation.

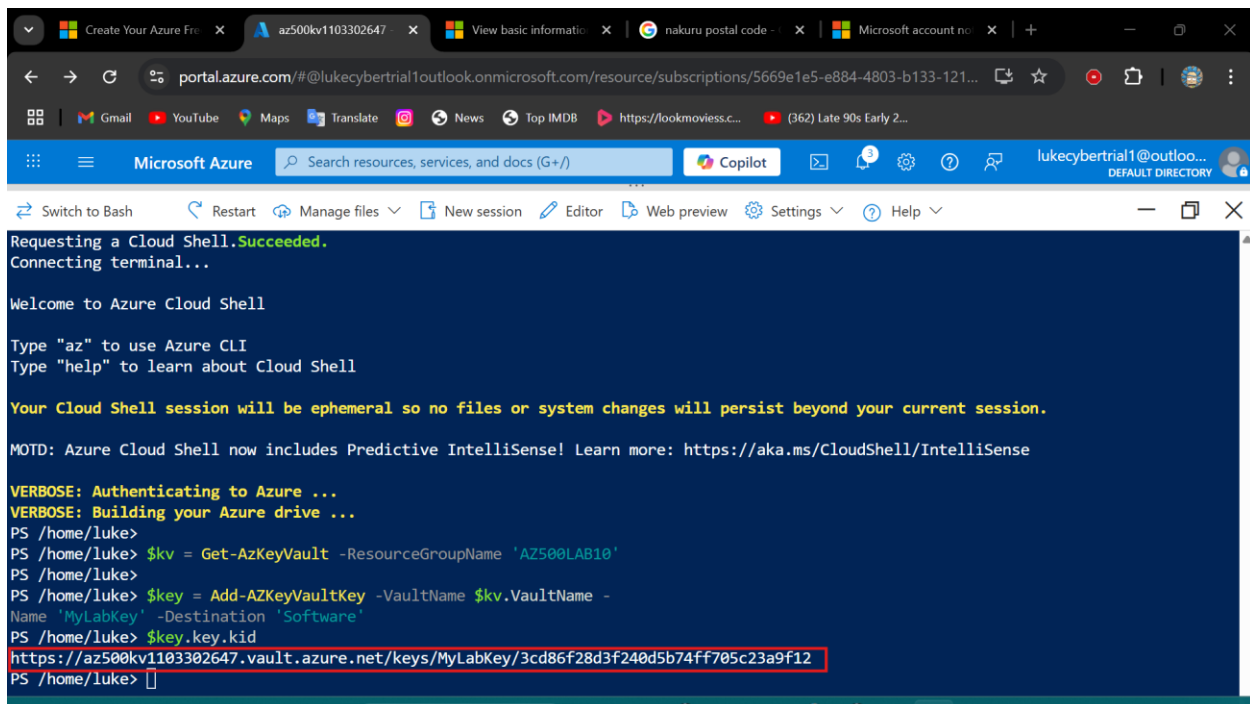
I began by opening **Azure Cloud Shell** via the Azure portal and ensured **PowerShell** was selected from the dropdown in the top-left corner of the Cloud Shell pane.

To retrieve my Key Vault object, I executed the following command:

`[$kv = Get-AzKeyVault -ResourceGroupName 'AZ500LAB10']`. Next, I added a new software-protected key named MyLabKey to the Key Vault: **`[$key = Add-AZKeyVaultKey -VaultName $kv.VaultName -Name 'MyLabKey' -Destination 'Software']`**.

To confirm the key was successfully created, I ran: **`[Get-AzKeyVaultKey -VaultName $kv.VaultName]`**.

Finally, I retrieved the unique identifier (Key Identifier) of the created key using: **`[$key.key.kid]`**. After verifying everything was in place, I minimized the Cloud Shell to continue with the lab.



```
Requesting a Cloud Shell.Succeeded.
Connecting terminal...

Welcome to Azure Cloud Shell

Type "az" to use Azure CLI
Type "help" to learn about Cloud Shell

Your Cloud Shell session will be ephemeral so no files or system changes will persist beyond your current session.

MOTD: Azure Cloud Shell now includes Predictive IntelliSense! Learn more: https://aka.ms/CloudShell/IntelliSense

VERBOSE: Authenticating to Azure ...
VERBOSE: Building your Azure drive ...
PS /home/luke> $kv = Get-AzKeyVault -ResourceGroupName 'AZ500LAB10'
PS /home/luke>
PS /home/luke> $key = Add-AZKeyVaultKey -VaultName $kv.VaultName -
Name 'MyLabKey' -Destination 'Software'
PS /home/luke> $key.key.kid
https://az500kv1103302647.vault.azure.net/keys/MyLabKey/3cd86f28d3f240d5b74ff705c23a9f12
PS /home/luke> []
```

Configure an Azure SQL database and a data-driven application

Enable a client application to access the Azure SQL Database service.

To enable my client application to access the Azure SQL Database service, I started by logging into the Azure portal. At the top, I used the search box labeled **Search resources, services, and docs** to type **App Registrations** and pressed Enter to navigate to the App Registrations page.

On the App Registrations blade, I clicked **+ New registration** to create a new application registration. On the **Register an application** page, I configured the following settings:

- **Name:** sqlApp
- **Redirect URI:** selected **Web** and entered https://sqlapp

After setting these details, I clicked **Register** to complete the process. This registered the application in Azure Active Directory, giving it an identity to authenticate the client application when accessing the Azure SQL Database.

Microsoft Azure Portal | Microsoft x Register an application - Microsoft x ac500v455024087.vault.azure.net x Error 400 when signing in - Micro x Extensions x +

portal.azure.com/#view/Microsoft_AAD_RegisteredApps/CreateApplicationBlade/quickStartType~/null/~/MSAApp~/false

Microsoft Azure Upgrade Search resources, services, and docs (G+?) Exploit kazyberthall@outlook.com

Register an application

* Name
The user-facing display name for this application (this can be changed later).

sqlApp

Supported account types
Who can use this application or access this API?

☒ Accounts in this organizational directory only (Default Directory only - Single tenant)
☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant)
☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)
☐ Personal Microsoft accounts only

Help me choose...

Redirect URI (optional)
We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.

Web https://sqlapp

Register an app you're working on here. Integrate gallery apps and other apps from outside your organization by adding from [Enterprise applications](#).

By proceeding, you agree to the [Microsoft Platform Policies](#) (?)

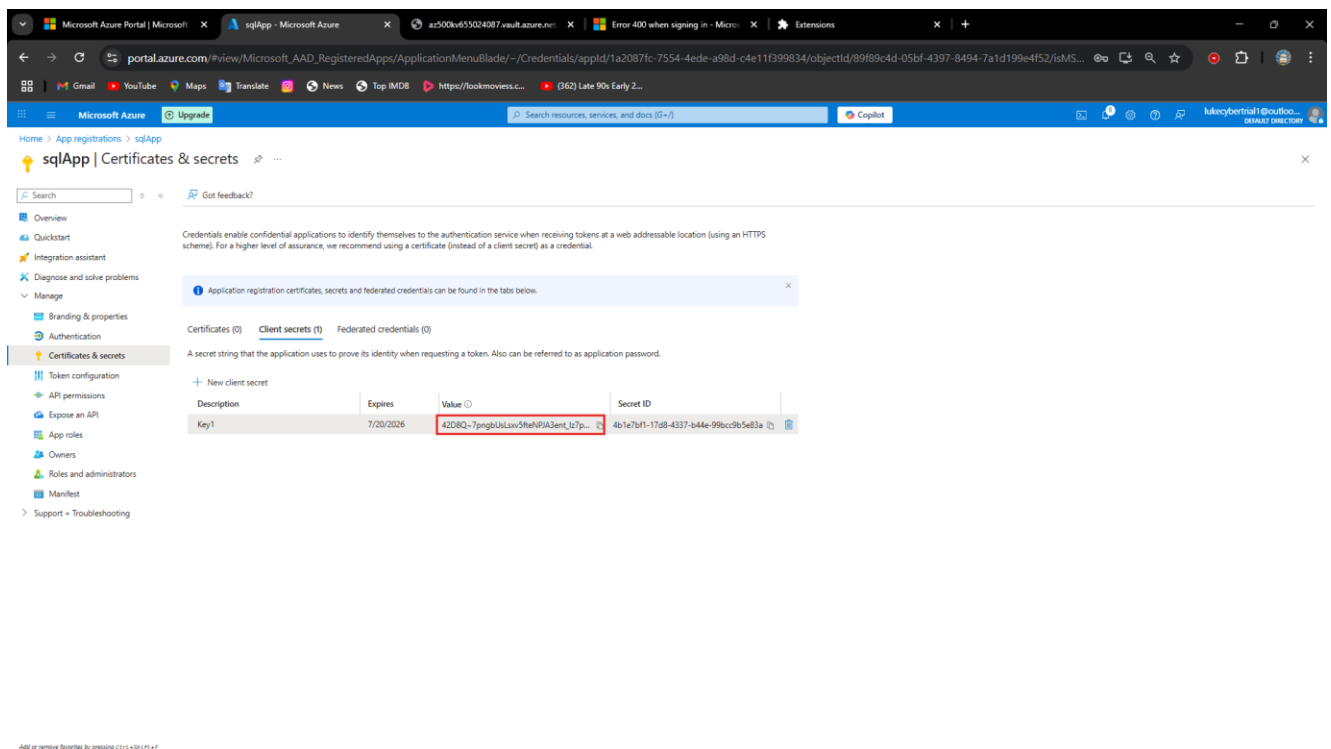
Register

On the **sqlApp** blade, I located and noted the **Application (client) ID** value, as this would be needed for the next task. Next, within the **Manage** section of the sqlApp blade, I clicked on **Certificates & secrets** to manage the application's authentication credentials. In the **Certificates**

& **secrets** blade, under the **Client Secrets** section, I clicked **+ New client secret** to create a new secret key for the application. In the **Add a client secret** pane, I specified the following:

- **Description:** Key1
- **Expires:** 12 months

After filling in these details, I clicked **Add** to generate and save the client secret, updating the application credentials needed for authenticating the client app. On the **sqlApp | Certificates & secrets** blade, identified the value of **Key1**.

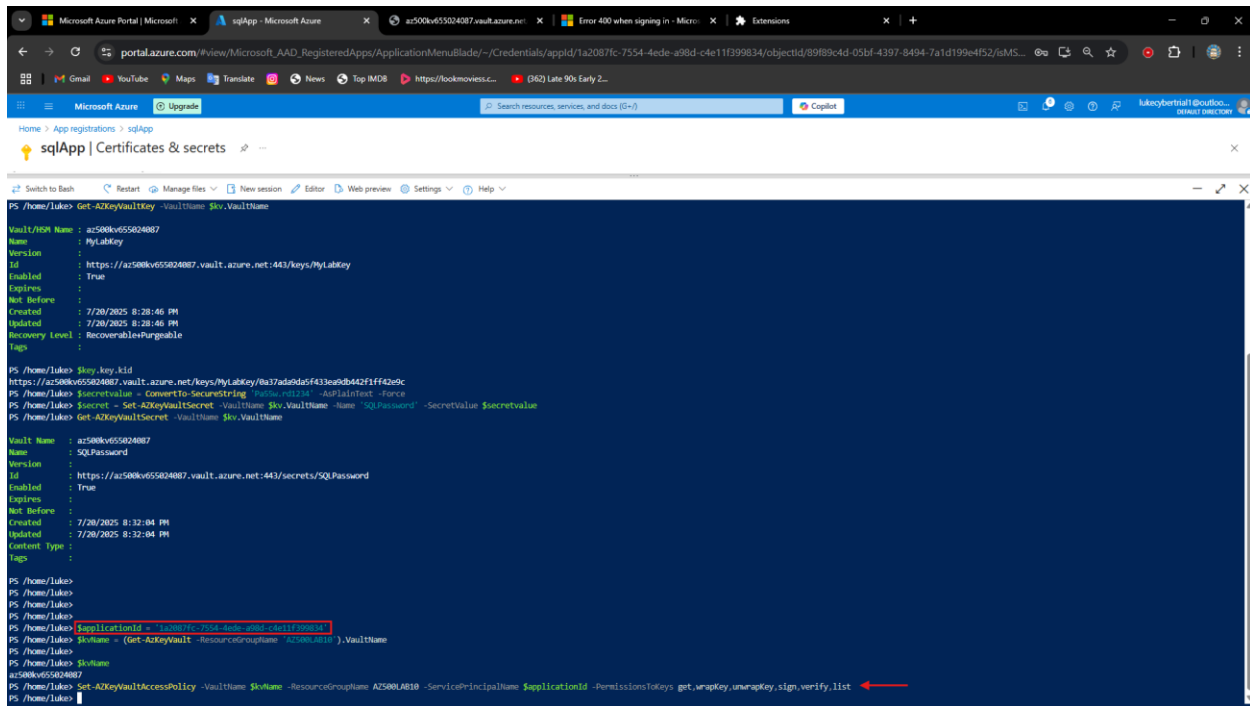


Create a policy allowing the application access to the Key Vault.

In this task, I granted the newly registered application permissions to access secrets stored in the Azure Key Vault. I started by opening a PowerShell session within the Cloud Shell pane of the Azure portal, ensuring that PowerShell was selected in the upper-left drop-down menu.

First, I created a variable to store the Application (client) ID I had recorded earlier by running: **[\$applicationId = '1a2087fc-7554-4ede-a98d-c4e11f399834']**. Next, I created a variable to store the Key Vault name by running: **[\$kvName = (Get-AzKeyVault -ResourceGroupName 'AZ500LAB10').VaultName]**, then, I granted the registered application permissions on the Key Vault by running: **[Set-AzKeyVaultAccessPolicy -VaultName \$kvName -ResourceGroupName**

AZ500LAB10 -ServicePrincipalName \$applicationId -PermissionsToKeys get,wrapKey,unwrapKey,sign,verify,list]



```
PS /home/luke> Get-AzKeyVaultKey -VaultName $kv.VaultName

Vault Name : az500k655024087
Name       : MyLabKey
Version    :
Id         : https://az500k655024087.vault.azure.net/keys/MyLabKey
Enabled    : True
Expires    :
Not Before :
Created    : 7/20/2025 8:28:46 PM
Updated    : 7/20/2025 8:28:46 PM
Recovery Level : Recoverable+Purgeable
Tags       :

PS /home/luke> $key.key.kid
https://az500k655024087.vault.azure.net/keys/MyLabKey/8a37ada8da5f433ea9db442f1ff42ebc
PS /home/luke> $secretValue = ConvertTo-SecureString "Password123" -AsPlainText -Force
PS /home/luke> Set-AzKeyVaultSecret -VaultName $kv.VaultName -Name "SQLPassword" -SecretValue $secretValue
PS /home/luke> Get-AzKeyVaultSecret -VaultName $kv.VaultName

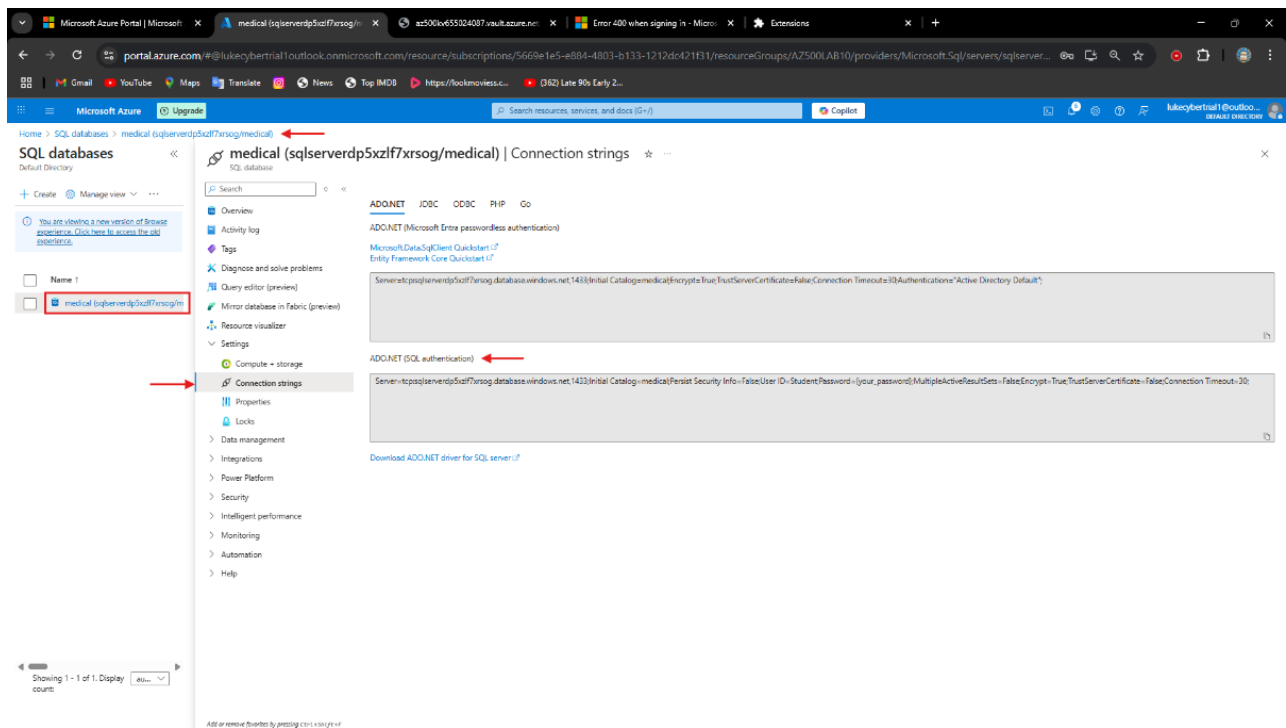
Vault Name : az500k655024087
Name       : SQLPassword
Version    :
Id         : https://az500k655024087.vault.azure.net/keys/SQLPassword
Enabled    : True
Expires    :
Not Before :
Created    : 7/20/2025 8:32:04 PM
Updated    : 7/20/2025 8:32:04 PM
Content Type :
Tags       :

PS /home/luke>
PS /home/luke>
PS /home/luke>
PS /home/luke>
PS /home/luke> $applicationId = "160877f-7554-46de-90ed-4a11f90834"
PS /home/luke> $kvName = (Get-AzKeyVault -ResourceGroupName "AZ500LAB10").VaultName
PS /home/luke> $kvName
az500k655024087
PS /home/luke> Set-AzKeyVaultAccessPolicy -VaultName $kvName -ResourceGroupName AZ500LAB10 -ServicePrincipalName $applicationId -PermissionsToKeys get,wrapKey,unwrapKey,sign,verify,list
PS /home/luke>
```

Retrieve SQL Azure database ADO.NET Connection String

I began by searching for **SQL databases** in the Azure portal's search box and pressing Enter. From the list of databases displayed, I selected the **medical** database that was provisioned by the ARM-template deployment in Exercise 1. I confirmed the deployment status by navigating to the Resource Group **AZ500LAB10** (or the resource group name I used), then checking the **Deployments** section in the settings pane to ensure the deployment had completed successfully.

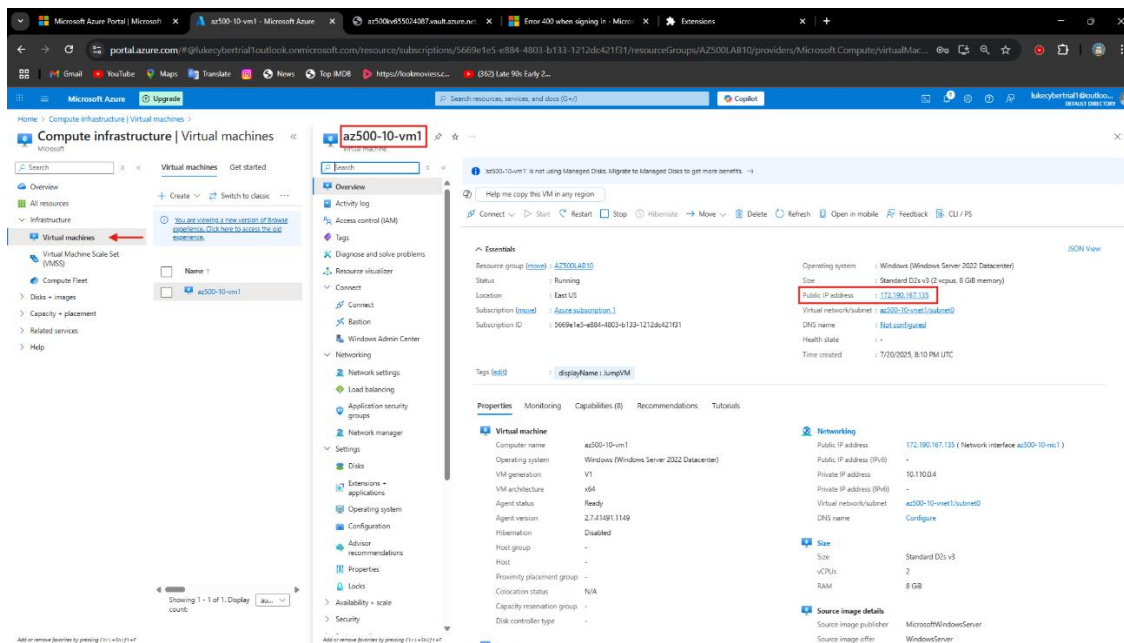
Once on the **medical** database blade, I went to the **Settings** section and clicked on **Connection strings**. Here, I reviewed various connection string formats including ADO.NET, JDBC, ODBC, PHP, and Go. I recorded the **ADO.NET (SQL authentication)** connection string as it would be needed later for connecting to the database. I noted to replace the {your_password} placeholder in the connection string with the actual password configured during the initial deployment.



Log on to the Azure VM running Visual Studio 2019 and SQL Management Studio 19

I began by verifying that the deployment initiated in Exercise 1 had completed successfully. To do this, I navigated to the **AZ500LAB10** resource group (or the name I chose) and checked the **Deployments** section in the settings pane.

Next, I searched for **Virtual Machines** using the Azure portal's search box and pressed Enter. From the list of virtual machines displayed, I selected the **az500-10-vm1** instance. On the **az500-10-vm1** blade, I noted the **Public IP address** displayed in the Essentials pane. This IP address would be required later to remotely connect to the VM hosting Visual Studio 2019 and SQL Server Management Studio 19.



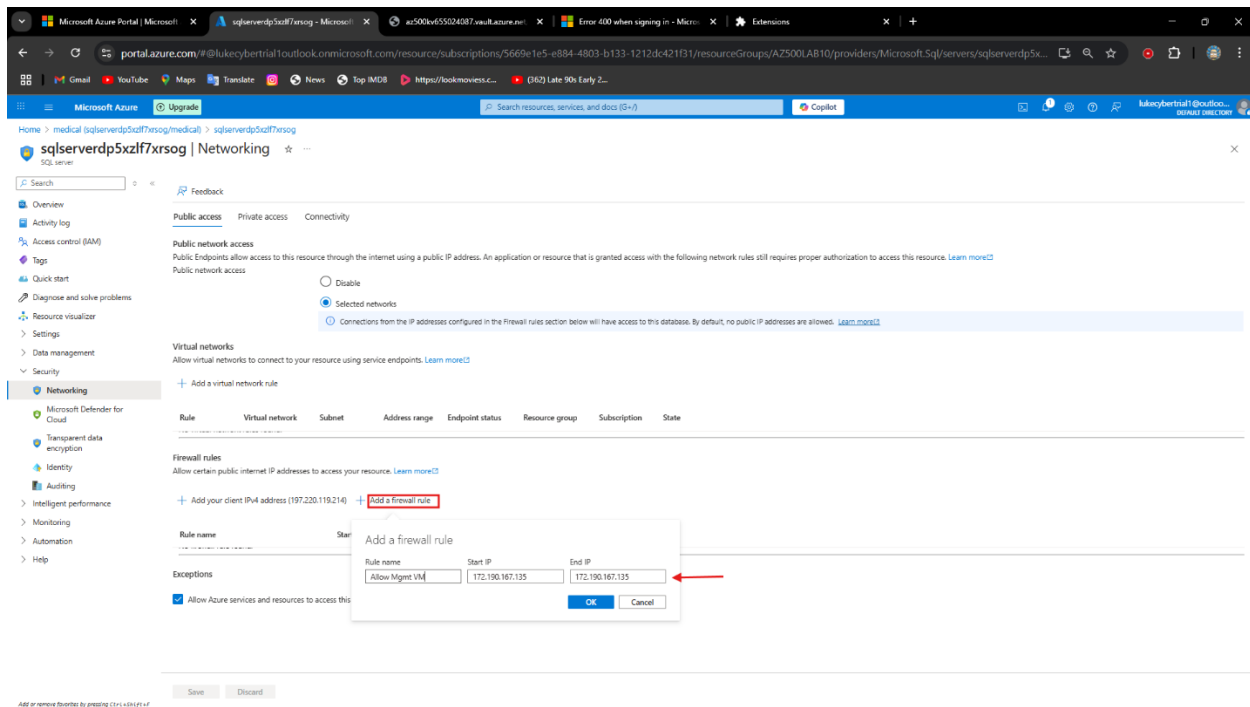
Create a table in the SQL Database and select data columns for encryption

I started by navigating to the **medical** SQL database blade in the Azure portal. In the Essentials section, I located and copied the **Server name**, knowing I would need this information later to connect to the database.

Next, I clicked **Set server firewall** on the toolbar to configure firewall rules for the SQL server. On the Firewall settings blade, I added a new firewall rule by clicking **+ Add a firewall rule**. I set the following:

- **Rule Name:** Allow Mgmt VM
- **Start IP:** 172.190.167.135
- **End IP:** 172.190.167.135

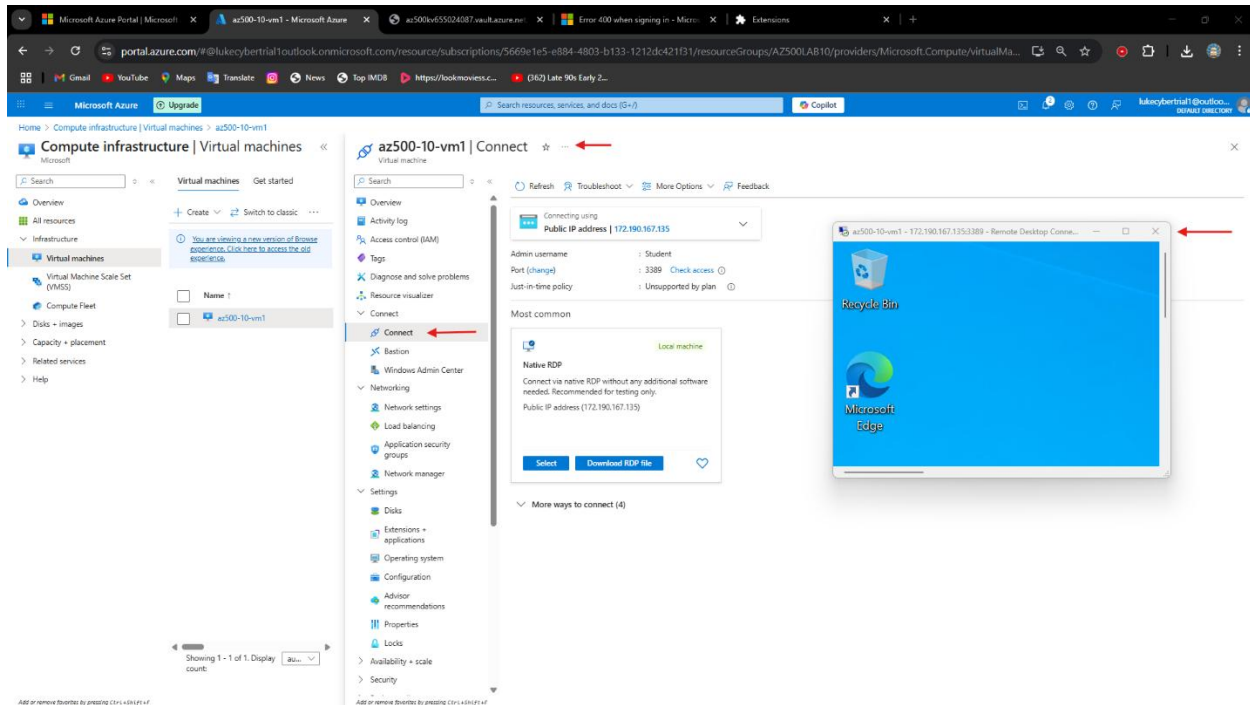
After entering these details, I clicked **Save** to apply the firewall rule, allowing the management VM to access the SQL server securely. The confirmation pane was then closed to proceed with the next steps.



I navigated back to the **az500-10-vm1** blade in the Azure portal and clicked on **Overview**. Then, I selected **Connect** from the toolbar, and from the drop-down menu, clicked **Connect** again. I downloaded the RDP file provided and used it to establish a Remote Desktop connection to the **az500-10-vm1** Azure VM. When prompted, I entered the credentials:

- **Username:** Student
- **Password:** Pa55w.rd

After successfully logging in, I waited for the Remote Desktop session and Server Manager to fully load. Once loaded, I closed the Server Manager to prepare the environment for the remaining lab steps, which would be performed inside the Remote Desktop session.

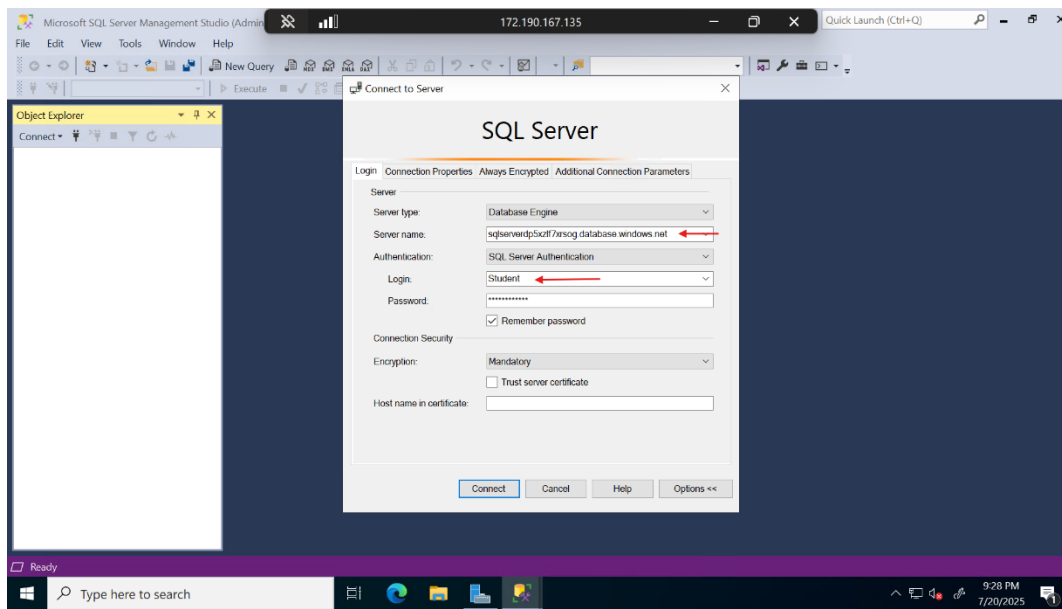


On the **az500-10-vm1** Azure VM, I installed SQL Server Management Studio (SSMS). After the installation, I launched SSMS to begin connecting to the Azure SQL Database.

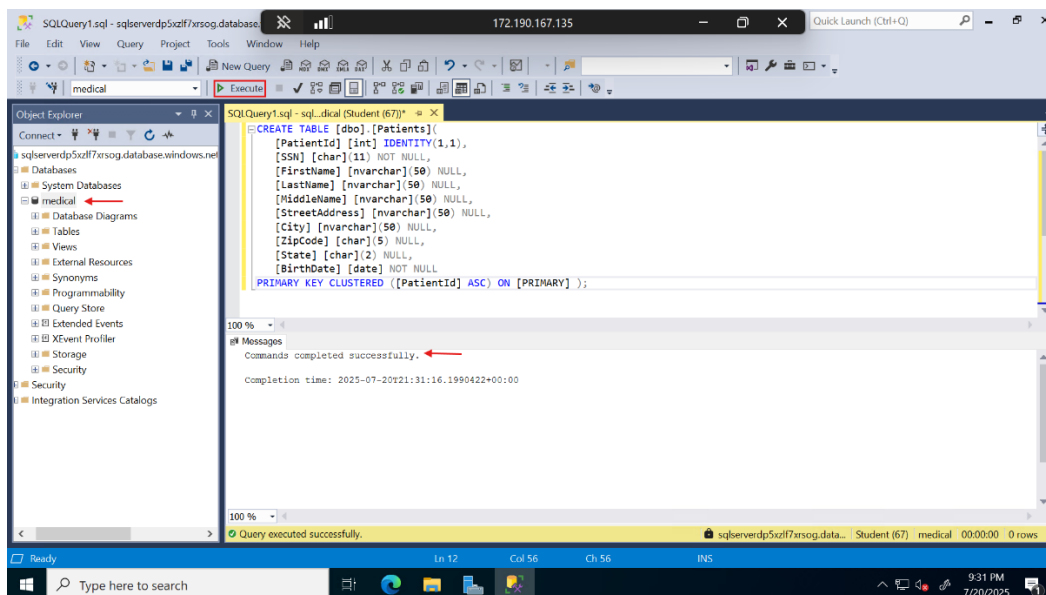
In the **Connect to Server** dialog box, I specified the following settings:

- **Server Type:** Database Engine
- **Server Name:** sqlserverdp5xzlf7xrsog.database.windows.net
- **Authentication:** SQL Server Authentication
- **Username:** Student
- **Password:** Pa55w.rd

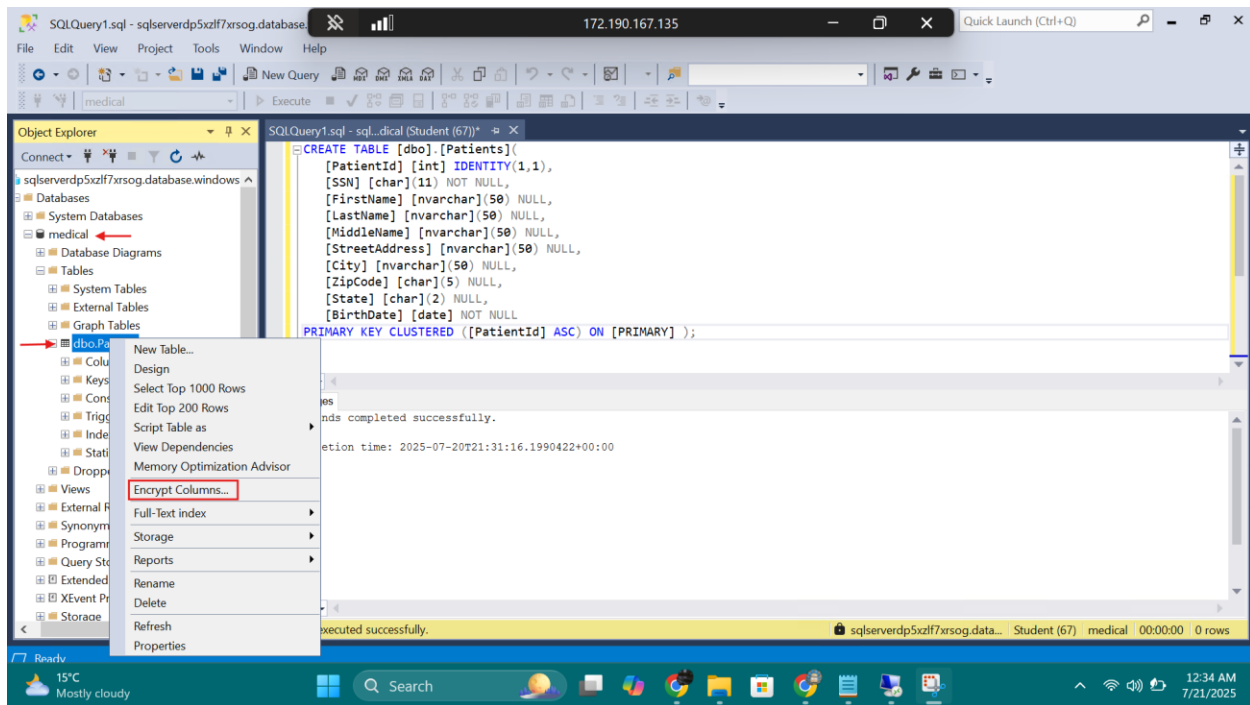
After entering these details, I clicked **Connect** to establish a connection to the SQL server and begin managing the database.



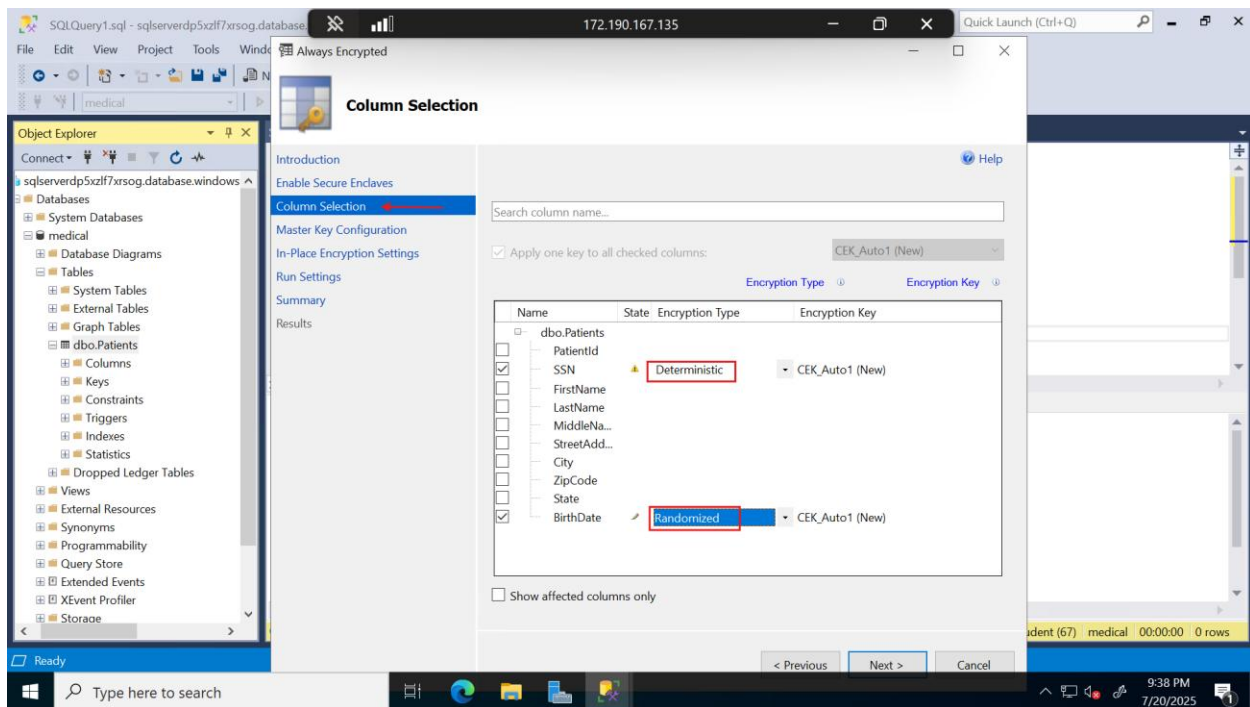
Within SQL Server Management Studio, I expanded the **Databases** folder in the Object Explorer pane. I then right-clicked the **medical** database and selected **New Query** to open a query window. I pasted the provided SQL code to create a **Patients** table, which includes columns for patient ID, SSN, name, address, and birthdate, with PatientId set as the primary key.



After verifying the code, I clicked **Execute** to create the table, once the table was successfully created, I navigated through the Object Explorer by expanding the **medical** database node, then the **Tables** node. I right-clicked the **dbo.Patients** table and selected **Encrypt Columns**, which launched the Always Encrypted wizard. On the wizard's Introduction page, I clicked **Next** to proceed with configuring encryption for the selected data columns.

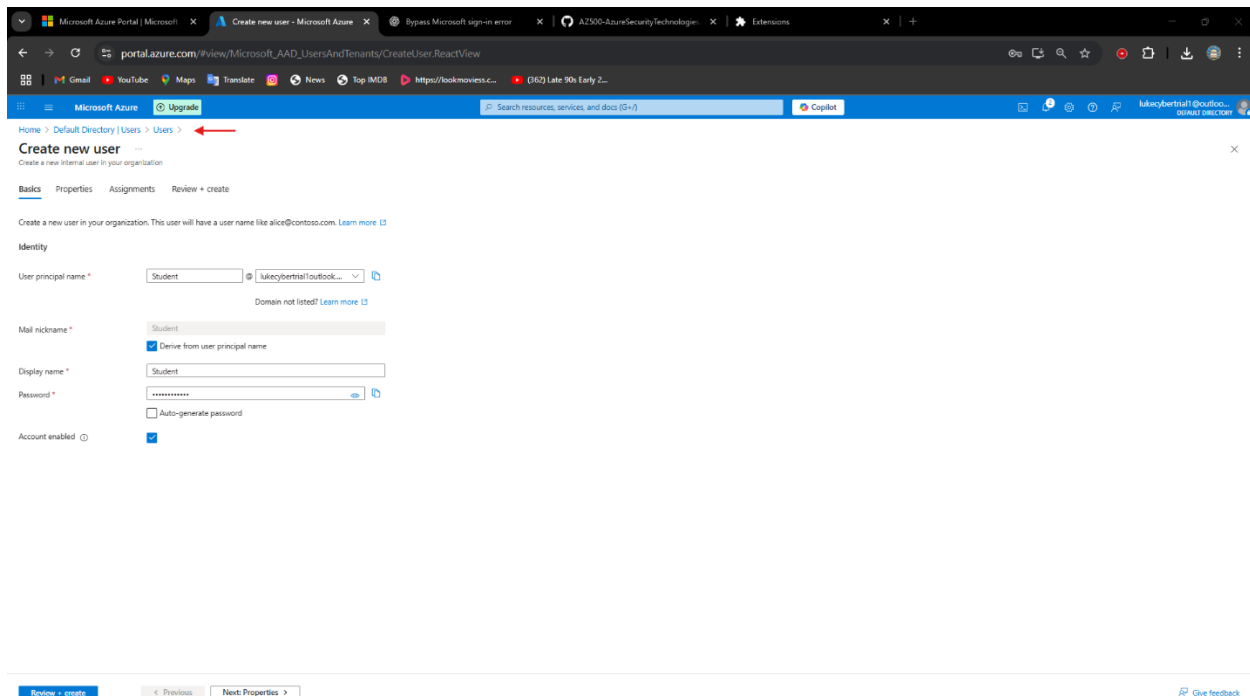


On the **Column Selection** page, I selected the **SSN** and **Birthdate** columns, set the **Encryption Type** of the **SSN** column to **Deterministic** and of the **Birthdate** column to **Randomized**, and click **Next**.



On the Master Key Configuration page of the Always Encrypted wizard, I selected **Azure Key Vault** as the key store provider. I then clicked **Sign in** and authenticated using the same user account that was used earlier to provision the Azure Key Vault in this lab.

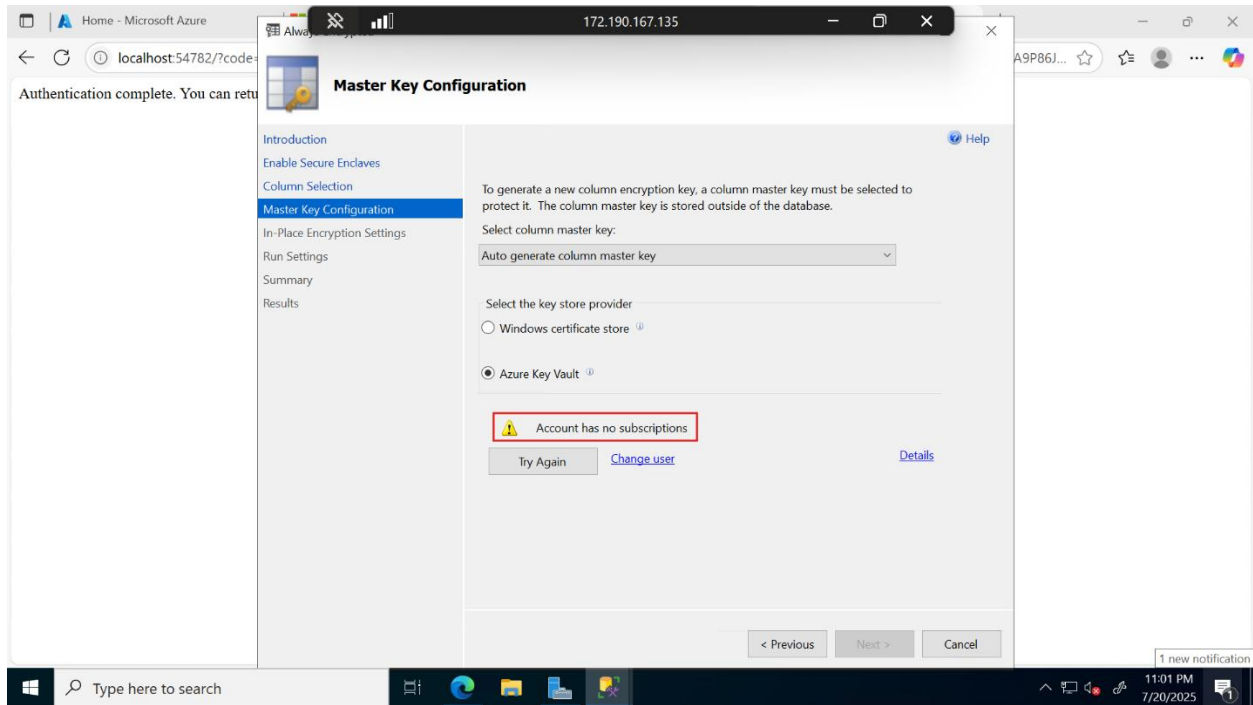
After signing in successfully, I ensured that the appropriate Key Vault appeared in the **Select an Azure Key Vault** drop-down list. Confirming this, I clicked **Next** to proceed. On the Run Settings page that followed, I simply clicked **Next** to continue with the encryption configuration.



On the Summary page of the Always Encrypted wizard, I clicked **Finish** to start the encryption process. When prompted, I signed in again using the same user account that I had used earlier to provision the Azure Key Vault. After the encryption process completed successfully, I reviewed the Results page and then clicked **Close** to exit the wizard, completing the encryption of the selected columns in the Patients table.

In SQL Server Management Studio, I expanded the **Security** and **Always Encrypted Keys** subnodes under the **medical** database in the Object Explorer pane. However, I encountered an issue stating that the account has no subscription, which prevented me from proceeding further with key management. At this point, I concluded the task due to this limitation.

The activity was designed to be completed using Azure Cloud Shell, but I encountered persistent difficulties accessing an account with the necessary subscription. Due to these issues, I was unable to proceed beyond a certain point and opted to conclude the exercise at that stage. Despite this setback, I fully understood the objectives and requirements of the assignment.



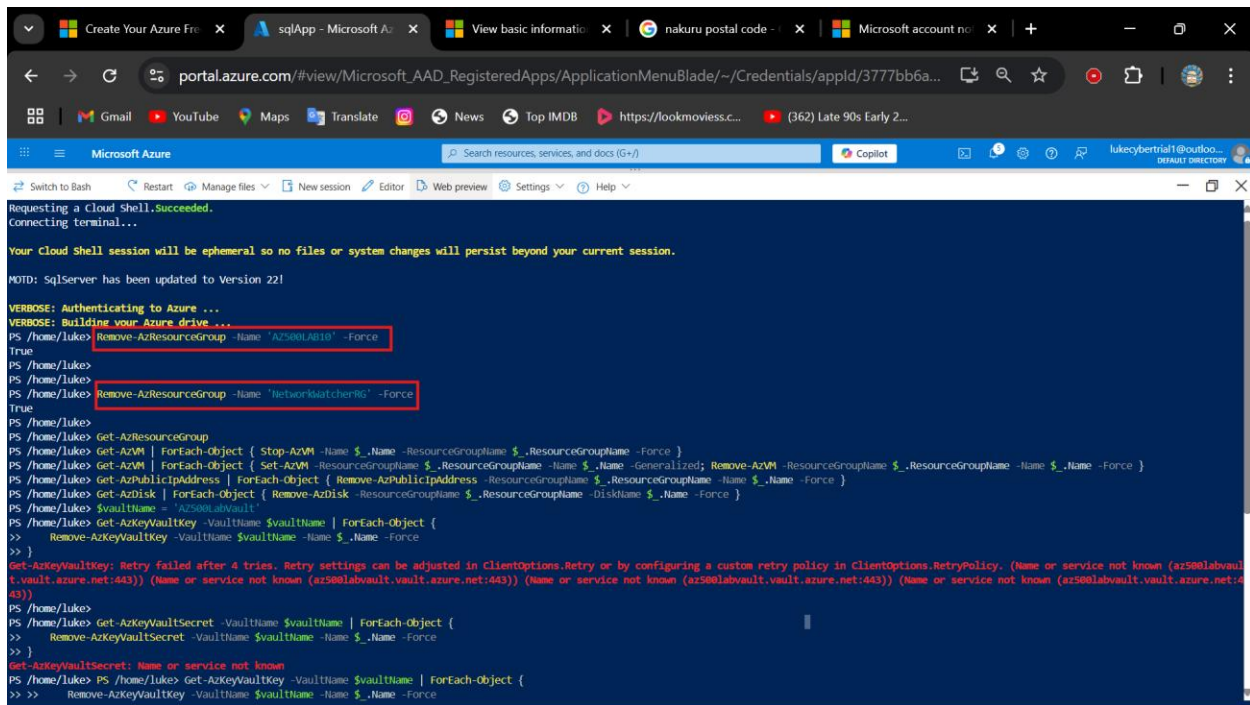
Resource Cleanup

After completing the deployment and Key Vault configuration tasks, I ensured all Azure resources were properly deleted to avoid incurring any further costs.

Using **Azure PowerShell in Cloud Shell**, I executed the following commands: **[Remove-AzResourceGroup -Name 'AZ500LAB10' -Force**

Remove-AzResourceGroup -Name 'NetworkWatcherRG' -Force]

This action deleted all resources associated with the lab—including virtual machines, networks, storage accounts, and the Azure Key Vault. By doing so, I confirmed that no chargeable services would remain active.



```
Requesting a Cloud Shell. Succeeded.
Connecting terminal...

Your Cloud Shell session will be ephemeral so no files or system changes will persist beyond your current session.

NOTED: SqlServer has been updated to Version 22!

VERBOSE: Authenticating to Azure ...
VERBOSE: Building your Azure drive ...
PS /home/luke> Remove-AzResourceGroup -Name 'AZ500LAB10' -Force
True
PS /home/luke>
PS /home/luke>
PS /home/luke> Remove-AzResourceGroup -Name 'NetworkWatcherRG' -Force
True
PS /home/luke>
PS /home/luke> Get-AzResourceGroup
PS /home/luke> Get-AZVM | ForEach-Object { Stop-AZVM -Name $_.Name -ResourceGroupName $_.ResourceGroupName -Force }
PS /home/luke> Get-AZVM | ForEach-Object { Set-AZVM -ResourceGroupName $_.ResourceGroupName -Name $_.Name -Generalized; Remove-AZVM -ResourceGroupName $_.ResourceGroupName -Name $_.Name -Force }
PS /home/luke> Get-AZPublicIpAddress | ForEach-Object { Remove-AZPublicIpAddress -ResourceGroupName $_.ResourceGroupName -Name $_.Name -Force }
PS /home/luke> Get-AZDisk | ForEach-Object { Remove-AZDisk -ResourceGroupName $_.ResourceGroupName -DiskName $_.Name -Force }
PS /home/luke> $vaultName = 'AZ500labVault'
PS /home/luke> Get-AzKeyVaultKey -VaultName $vaultName | ForEach-Object {
>> Remove-AzKeyVaultKey -VaultName $vaultName -Name $_.Name -Force
>> }
Get-AzKeyVaultKey: Retry failed after 4 tries. Retry settings can be adjusted in ClientOptions.Retry or by configuring a custom retry policy in ClientOptions.RetryPolicy. (Name or service not known (az500labvault.azure.net:443)) (Name or service not known (az500labvault.azure.net:443)) (Name or service not known (az500labvault.azure.net:443)) (Name or service not known (az500labvault.azure.net:443))
PS /home/luke>
PS /home/luke> Get-AzKeyVaultSecret -VaultName $vaultName | ForEach-Object {
>> Remove-AzKeyVaultSecret -VaultName $vaultName -Name $_.Name -Force
>> }
Get-AzKeyVaultSecret: Name or service not known
PS /home/luke> PS /home/luke> Get-AzKeyVaultKey -VaultName $vaultName | ForEach-Object {
>> Remove-AzKeyVaultKey -VaultName $vaultName -Name $_.Name -Force
>> }
```

Exercise Summary

This lab exercise involved working with core Azure security and deployment tools. It was divided into multiple key tasks that helped demonstrate real-world skills in setting up secure environments in Microsoft Azure:

1. Deployment of Azure Resources:

- A Virtual Machine (VM) was deployed using a custom ARM template via the Azure Portal.
- The VM came pre-installed with **Visual Studio 2019** and **SQL Server Management Studio 19** to simulate a developer-friendly environment.

2. Creation and Configuration of Azure Key Vault:

- An Azure Key Vault was created using PowerShell in Azure Cloud Shell.
- Role-based access was disabled initially, and manual **access policies** were added.
- Permissions for Keys, Secrets, and Certificates were granted to the signed-in user.

3. Adding and Managing Keys:

- A software-protected key named MyLabKey was created.

- Commands were run to verify the key creation and display its unique identifier.
- The Key Vault blade in the Azure Portal was used to visually inspect the created key and its metadata.

4. Resource Cleanup:

- To avoid unintended billing, all resources were deleted using the following commands: ***[Remove-AzResourceGroup -Name 'AZ500LAB10' -Force
Remove-AzResourceGroup -Name 'NetworkWatcherRG' -Force]***
- This step ensured all related Azure services (including the VM, Key Vault, storage, and networking components) were fully removed.

Conclusion

This lab successfully demonstrated how to securely deploy and manage Azure services, including virtual machines and key vaults. Through the hands-on experience, I gained insight into using ARM templates, PowerShell scripting in Cloud Shell, and configuring granular access permissions within Key Vault. By completing proper resource cleanup at the end, I also reinforced best practices for managing cloud resources cost-effectively.